

포트폴리오

K-POP Radar 2.0 / Blip

[사전 집계](#) · [캐시](#) · [인덱스 최적화](#)

스페이스오디티 · 2025.01 - 2026.01

글로벌 K-POP 데이터 분석 서비스

NestJS

TypeORM

PostgreSQL

GCP Firestore

Redis

Kubernetes

Argo CD

- ✓ 기존 Blip 서비스의 MySQL 운영 데이터와 신규 K-POP Radar의 PostgreSQL 구조가 공존하는 환경에서 데이터 동기화 흐름과 조회 모델 재설계
- ✓ 화면에서 자주 호출되는 지표는 사전 집계 구조로 분리하고 탐색형 데이터만 동적 조회로 남겨 대시보드 응답 안정성과 운영 복잡도 사이의 균형 확보
- ✓ 수집 결과는 Firestore에 Raw 형태로 저장하고 db-syncer를 통해 PostgreSQL 기준 모델로 반영하는 구조 운영
- ✓ 중간 집계 상태와 데이터 정합성을 확인하는 Slack 알림 로직 구성

문제

기존 Blip 서비스의 운영 데이터는 MySQL에 계속 쌓이고 있었고 새로 구축하는 K-POP Radar는 PostgreSQL 기반으로 설계해야 했기 때문에 한동안 데이터베이스가 이원화된 상태였습니다. 단순히 데이터를 옮기는 문제가 아니라 새 서비스 화면 구조와 기획 의도에 맞는 PostgreSQL 기준 데이터 모델을 다시 설계해야 했고 핵심은 기존 데이터를 그대로 이전하는 것이 아니라 서비스에서 실제로 사용하는 지표를 기준으로 조회 구조를 다시 정의하는 일이었습니다.

해결

화면에서 자주 호출되고 계산 비용이 큰 지표는 사전 집계 테이블로 분리하고 비교나 탐색 성격이 강한 데이터만 동적 조회로 남겼습니다. 실제 쿼리 패턴 기반 인덱스 최적화, 시계열 데이터 파티셔닝,

Redis 캐시를 함께 적용해 조회 병목을 줄였습니다. 운영 단계에서는 동기화 누락, 집계 지연, 이상치를 빠르게 확인할 수 있도록 모니터링과 Slack 알림 구조를 함께 두었습니다.

아키텍처

Backend: NestJS, TypeORM / Database: PostgreSQL, GCP Firestore, Redis / Infra: Kubernetes, Argo CD

 더 자세한 내용 보기 →

두디스 (Dothis)

응답속도 40초 → 3초

두디스 프로젝트 · 2023.10 – 2024.11 Co-founder

유튜브 콘텐츠 소재 · 키워드 트렌드 분석 플랫폼

NestJS

TypeORM

MariaDB

OpenSearch

Kafka

Logstash

Redis

- ✓ MySQL 조인 중심 구조에서 발생하던 병목을 분석해 비정규화 기반 검색·집계 구조로 전환하고 핵심 집계 API 응답 속도를 약 40초에서 3초 수준으로 개선
- ✓ 검색 요청은 토큰 테이블에서 후보 콘텐츠를 먼저 좁히고 이후 OpenSearch의 비정규화 데이터에서 최종 조회하는 방식으로 구성해 탐색 비용과 응답 지연을 함께 축소
- ✓ BERT 기반 텍스트 임베딩·토큰나이징·클러스터링 파이프라인 구축

문제

초기에는 MySQL에서 크롤링 데이터를 조인하고 집계해 API를 제공했습니다. 데이터가 쌓이면서 병목이 심해졌고 핵심 집계 API 하나를 호출하면 40초에서 1분까지 걸리는 상황이 발생했습니다. 문제는 단순히 느린 쿼리 하나가 아니었습니다. 조인 중심 구조 자체가 조회 목적과 맞지 않았고 시간이 갈수록 한계가 더 분명해졌습니다.

해결

처음에는 OpenSearch 기반 구조를 도입해 검색과 집계 워크로드를 분리했지만 구조적으로 완전히 해결된 수준은 아니었습니다. 결국 문제의 핵심은 조인을 빠르게 하는 것이 아니라 조인을 계속 요구하는 구조 자체에 있다고 판단했습니다. DWH 데이터를 Logstash로 비정규화해 OpenSearch에 인덱싱하고 조회 시점 조인을 최대한 없애는 방식으로 전환했습니다. 검색 요청은 토큰 테이블에서 키워드와 연결된 콘텐츠 ID를 먼저 찾고 그 후보군을 기준으로 OpenSearch의 비정규화 데이터에서 최종 조회하도록 설계했습니다.

아키텍처

Main: MySQL / DWH: MariaDB / 인덱싱: Logstash → OpenSearch / 캐시: Redis, 토큰 테이블 / 수집: Kafka

 더 자세한 내용 보기 →

프린트시티 lamDesign

Optimistic Locking · 상품 페이지 자동 생성

프린트시티 · 2021.07 – 2023.10

인쇄물 커머스 플랫폼 자동화 시스템

NestJS

Next.js

MongoDB

AWS EC2/ECS

GitHub Actions

- ✓ 하드코딩으로 운영하던 상품 상세 페이지 구조를 조합 데이터 기반 자동 생성 방식으로 전환해 상품 추가와 수정이 반복될수록 커지던 운영 비용 절감
- ✓ Next.js 공통 템플릿과 MongoDB 조합형 JSON 구조를 바탕으로 관리자 화면에서 상품 정의만으로 페이지 구성이 달라지는 상품 상세 구조 설계
- ✓ 재고 차감에는 Optimistic Locking을 적용해 동시 주문 환경에서도 데이터 정합성 확보
- ✓ 신규 개발팀의 개발 컨벤션과 배포 프로세스 표준화 정리

문제

기존 구조는 상품 하나당 상세 페이지 하나를 직접 하드코딩하는 방식이었습니다. 상품이 늘어날수록 페이지 수가 같이 늘었고 수정 범위도 커졌습니다. 운영 변경이 생길 때마다 개발이 같이 들어가야 했고 레거시 파일 복사 기반 유지보수 때문에 휴먼 에러도 자주 발생했습니다. 여기에 주문이 동시에 들어오면 재고 불일치 문제가 생겼습니다.

해결

상품을 페이지 단위가 아니라 조합 데이터 단위로 다시 정의했습니다. 관리자 페이지에서 원재료와 옵션을 조합해 MongoDB document로 관리하고 Next.js 공통 템플릿이 그 JSON 구조를 바탕으로 조건부 렌더링되도록 설계했습니다. 재고 관리에는 Optimistic Locking을 적용해 버전 기반으로 충돌을 감지하고 재시도 가능한 흐름을 설계해 동시 주문 환경에서도 정합성을 유지하도록 했습니다.

아키텍처

Backend: NestJS / Frontend: Next.js / Database: MongoDB / Infra: AWS EC2, ECS / CI/CD: GitHub Actions

 더 자세한 내용 보기 →

사이드 프로젝트

개인 투자 어시스턴트 플랫폼 (Stock Pile)

[GitHub](#)[Live](#)

2026.04 - 진행중

자연어 챗봇으로 매매를 기록하고 LLM 기반 종목 분석 리포트를 제공하는 개인 투자 관리 서비스 (pnpm Turborepo 모노레포)

데모 계정 이메일: demo@byeongguk.cloud 비밀번호: Test1234!

[TypeScript](#)[NestJS](#)[Next.js](#)[PostgreSQL\(pgvector\)](#)[Redis](#)[Docker](#)[GitHub Actions](#)

- Groq / Anthropic 멀티 프로바이더 추상화 - 공급사 전환 시 호출부 변경 없이 Provider 교체 가능한 어댑터 설계
- 챗봇 2-step 파이프라인: 의도 분류 (TRADE_ENTRY / INVESTMENT_QUERY) → 종목 추출 후 매매 파싱 또는 투자 어드바이저로 라우팅
- DART 재무 · 네이버 뉴스 · Yahoo Finance 지표를 Promise.all 병렬 수집 후 LLM 합성 리포트 생성, 결과를 Redis에 24h 캐싱하여 API 호출 비용 절감
- pgvector 기반 뉴스 임베딩 RAG - 코사인 유사도 검색으로 분석 컨텍스트 보강
- shared-types · db-schema 패키지를 단일 소스로 관리해 DTO 중복 제거

차량 운행 로그 분석 파이프라인

[GitHub](#)

2026.04

차량 Raw 로그를 실시간 수신·정제하여 위험 운전 패턴을 탐지하는 백엔드 파이프라인

[Python](#)[FastAPI](#)[Kafka](#)[PostgreSQL](#)[NumPy](#)[Docker](#)

- Kafka 토픽 구독 → 10초 슬라이딩 윈도우 버퍼링 후 일괄 처리로 건별 DB I/O 제거, 처리량 향상

- `np.interp` + NumPy 벡터화 `haversine`으로 결측값 보간 및 거리 계산 – 순수 Python 루프 대비 처리 속도 개선
- Bounding box 사전 필터로 제한구역 비교 연산 $O(N \times M)$ 의 후보를 사전 축소, 불필요한 정밀 계산 회피
- SQLAlchemy 2.0 Core bulk insert로 대용량 GPS 레코드 적재 최적화
- SHA-256 기반 Trip 역등성 처리 – 재전송 시 중복 적재 방지
- MAX_RECORDS 가드 + Pydantic field_validator로 OOM 및 입력 이상치 방어

제가 기여할 수 있는 방식

→ 느린 조회·집계 구조 개선

조회 패턴과 데이터 모델을 함께 분석해
병목 원인을 찾고 필요한 경우 비정규화
·캐시·집계 구조 개선으로 성능을 높일
수 있습니다.

→ 데이터 수집·가공 파이프라인 안정화

스크래핑, 적재, 가공, 집계 흐름을 분리
해 운영 중 오류를 빠르게 확인하고 재
처리 가능한 구조로 개선할 수 있습니
다.

→ 요구사항을 API와 데이터 구조로 구체
화

기획·운영 요구를 단순 기능 단위가 아
니라 데이터 흐름과 사용 시나리오 기준
으로 정리해 실제 개발 가능한 형태로
풀어낼 수 있습니다.

→ 운영 이슈를 구조 개선으로 연결

장애나 성능 저하가 발생했을 때 임시
대응에 그치지 않고 재발 가능성을 줄이
는 방향으로 구조를 개선하는 데 강점이
있습니다.

→ 유지보수 가능한 코드베이스 정리

테스트, 문서화, 책임 분리를 통해 다른
개발자도 이해하고 이어서 개발할 수 있
는 코드 구조를 만드는 데 기여할 수 있
습니다.

→ 반복 업무 생산성 향상

반복 구현, 테스트 초안, 리팩토링 검토
단계에서는 AI 코딩 도구를 활용해 속도
를 높이되 최종 구조와 품질은 직접 검
증하는 방식으로 개발할 수 있습니다.

유병국

Backend Developer

Velog GitHub Portfolio Email